

Software Architecture Document

FailSafe AI – Student Success Intelligence Platform

Project:	FailSafe AI
Version:	1.0
Prepared For:	Bharat Academix CodeQuest 2026
Team Name:	Naanga Moonuperu

1. Introduction

This Software Architecture Document (SAD) outlines the structural design, components, and constraints of **FailSafe AI**, an intelligent academic support platform. The document defines the scope of the system, technical capabilities, and how the machine learning intelligence integrates with the user interface to deliver real-time risk predictions.

Target Audience: CodeQuest 2026 Judges, Developers, and Technical Reviewers.

2. System Overview & Context

FailSafe AI operates as a unified web application allowing two primary personas—Students and Faculty—to interact with academic data. The system takes in academic metrics (grades, absences, study hours) and utilizes a Random Forest machine learning model to predict student failure risk.

- **External Interfaces:** End-user web browsers.
- **Core System:** Python Streamlit server managing UI rendering and orchestrating data flow.
- **Intelligence Engine:** Pre-trained scikit-learn models serialized via Pickle.

3. Architectural Views

3.1 Logical View

The logical architecture is modular, separating UI concerns from data processing and machine learning inference:

- **UI Layer (`app.py`):** Orchestrates the page layout, captures user input via Streamlit widgets, and displays visual insights.
- **Component Library (`components.py`):** Reusable UI fragments (metrics tiles, risk gauges, custom SVGs) ensuring design consistency.
- **Inference Engine (`model.pk1`):** A serialized Random Forest classifier handling the core prediction logic.

3.2 Process View

The execution thread follows a stateless HTTP request-response cycle:

1. User inputs parameters via the sidebar form in the UI.
2. A Pandas DataFrame is dynamically generated with features [`G1`, `G2`, `absences`, `studytime`, `failures`].
3. The DataFrame is passed to the `predict_proba()` method of the loaded Random Forest model.
4. The model returns a risk label (Low/Medium/High) and confidence percentage.
5. UI components dynamically update to reflect the results using Plotly graphs and HTML/CSS components.

3.3 Deployment View

FailSafe AI is designed for single-node deployment, ensuring low latency and easy hosting.

- **Runtime:** Python 3.10+
- **Web Server:** Streamlit's native Tornado-based server.
- **Hosting:** Can be deployed locally via command line or remotely via Streamlit Community Cloud / Docker containers.

4. Functional Requirements

- **Risk Prediction:** System must accurately classify a student's risk of failure based on 5 quantitative metrics.
- **Dynamic Rendering:** System must generate an Academic Health Score, personalized recovery plans, and department-specific skill hubs.
- **Faculty Mode:** System must allow educators to quickly screen students and suggest mentor interventions.
- **State Management:** Analysis results must persist across UI updates (sliders/buttons) using session state.

5. Non-Functional Attributes

- **Performance:** Model inference must execute in under 100 milliseconds to provide a real-time interactive experience.
- **Usability:** UI must be responsive, applying the custom design system (`style.css`) with dark mode contrast and accessibility.
- **Reliability:** Risk calculations must be deterministic and clamped between 0-100 to prevent edge-case scoring errors.

6. Data Architecture

- **Training Data:** Sourced from Math and Portuguese student performance datasets (`student-mat.csv`, `student-por.csv`).
- **Feature Engineering:** Selected 5 critical features: G1 (Period 1 Grade), G2 (Period 2 Grade), absences, studytime, failures.
- **Storage:** No external database is required. The model is pre-trained offline via `train_model.py` and deployed as a read-only serialized binary artifact (`model.pkl`).

7. Design Constraints & Dependencies

- **Framework Constraint:** Built entirely on Streamlit, limiting complex client-side JavaScript execution. Custom UI is injected via HTML/CSS (`unsafe_allow_html=True`).
- **Dependency Versions:** Requires specific library versions (e.g., `scikit-learn >= 1.3`, `pandas >= 1.5`) to ensure pickle file compatibility during deserialization.
- **Stateless Nature:** The model does not dynamically retrain on new user inputs; model updates require offline execution of the training pipeline.